

COMSATS UNIVERSITY ISLAMABAD

WAH CAMPUS

Department of Computer Science

DevOps Graded Assignment #01

Course Title: DevOps

Course Code: CSC323

Instructor Name: Sir Ikram

Class: BSE-8A

Name	Registration No.
Muhammad Sami	SP23-BSE-039

Lab Assignment 01: Docker Installation and Containerization of a 3-Tier Laravel Application

This report documents the practical work carried out for the DevOps lab assignment. The assignment is divided into three parts: installing and configuring Docker on a local virtual machine, containerizing a Laravel-based "Student Management System" as a three-tier application (frontend, backend, and database), and verifying that the containers can communicate with each other correctly. Each step below first explains the activity and the command used, and is then followed by the terminal screenshot that was captured as evidence of that step.

Part A: Installing and Configuring Docker

Before any containerization work can begin, Docker Engine and the Docker Compose plugin must be installed and verified on the local virtual machine (Ubuntu 25.04, hostname osboxes, user samikhan). The following steps walk through this installation from a clean system to a fully working Docker setup.

Checking System Information and Docker Availability

The first step is to confirm basic details about the operating system and CPU architecture of the virtual machine, and to check whether Docker and Docker Compose are already available on the system.

Command Used:

```
cat /etc/os-release
uname -m
docker --version
docker compose version
```

Command Explanation:

- cat /etc/os-release → prints the operating system name and version so the correct installation instructions can be chosen.
- uname -m → prints the machine's hardware architecture (x86_64), which determines which Docker package build is required.
- docker --version and docker compose version → check whether Docker and the Compose plugin are already installed.

What Is Being Done: The student is verifying the environment before installing anything. This is a standard first step in any DevOps setup, since the installation commands for Docker depend on the Linux distribution and version being used.

Observed Result: The output confirms the VM is running Ubuntu 25.04 ("Plucky Puffin") on an x86_64 machine, and that Docker is not yet installed — the shell reports that the docker command was not found but can be installed with "sudo apt install docker.io". This confirms that a fresh Docker installation is required.

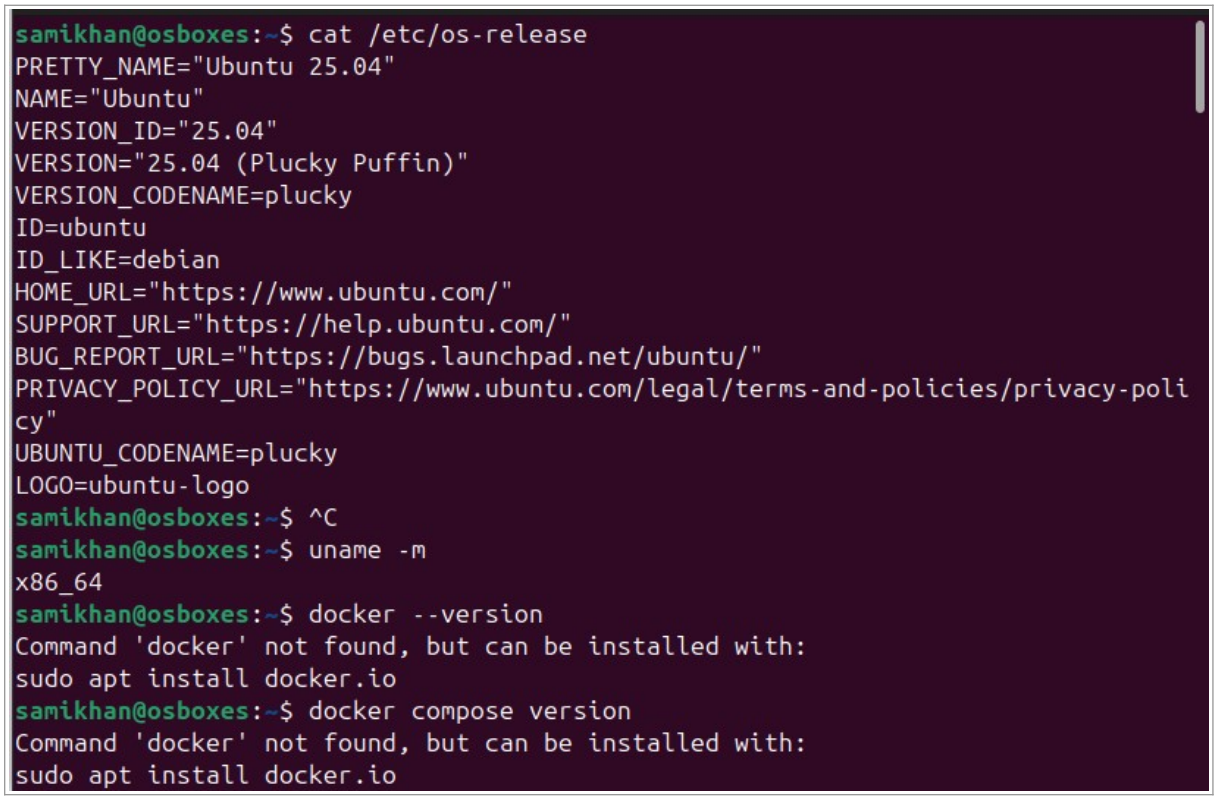


Figure 1: Checking the operating system, architecture, and current Docker availability.

Installing Prerequisite Packages

Docker's official installation method requires a small set of prerequisite packages to be present so that the Docker APT repository can be added securely.

Command Used:

```
sudo apt install -y ca-certificates curl
```

Command Explanation:

- sudo apt install -y → installs packages non-interactively using administrator privileges.
- ca-certificates → provides trusted SSL certificates needed to connect securely to Docker's download servers.
- curl → a command-line tool used to download files (such as Docker's GPG key) from the internet.

What Is Being Done: The student is preparing the system so that packages can later be downloaded securely from Docker's official repository.

Observed Result: The output shows that ca-certificates was already up to date and that curl was successfully downloaded and installed, confirming the system is ready for the next step.

```
samikhan@osboxes:~$ sudo apt install -y ca-certificates curl
[sudo] password for samikhan:
ca-certificates is already the newest version (20241223).
ca-certificates set to manually installed.
Installing:
  curl

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
  Download size: 258 kB
  Space needed: 503 kB / 485 GB available

Get:1 http://archive.ubuntu.com/ubuntu plucky/main amd64 curl amd64 8.12.1-3ubun
tu1 [258 kB]
Fetched 258 kB in 4s (70.7 kB/s)
Selecting previously unselected package curl.
(Reading database ... 186777 files and directories currently installed.)
Preparing to unpack .../curl_8.12.1-3ubuntu1_amd64.deb ...
Unpacking curl (8.12.1-3ubuntu1) ...
Setting up curl (8.12.1-3ubuntu1) ...
Processing triggers for man-db (2.13.0-1) ...
```

Figure 2: Installing the ca-certificates and curl prerequisite packages.

Adding the Docker Repository and Installing Docker Engine

With the prerequisites in place, the official Docker APT repository is added to the system so that the genuine Docker Engine packages (rather than the older distribution-provided docker.io package) can be installed.

Command Used:

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
 -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo \
 "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
 https://download.docker.com/linux/ubuntu \
 $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" | \
 sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
 docker-buildx-plugin docker-compose-plugin
```

Command Explanation:

- sudo install -m 0755 -d /etc/apt/keyrings → creates a directory to store trusted GPG keys with the correct permissions.
- curl … -o /etc/apt/keyrings/docker.asc → downloads Docker's official GPG signing key so that packages from the repository can be verified as authentic.
- echo "deb …" | sudo tee /etc/apt/sources.list.d/docker.list → registers Docker's official APT repository, matching it to the exact Ubuntu codename (plucky) detected from /etc/os-release.
- sudo apt update → refreshes the package index so the newly added repository is recognised.
- sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin → installs the Docker Engine, its command-line interface, the containerd runtime, and the Buildx and Compose plugins.

What Is Being Done: The student is switching from the outdated Ubuntu-provided Docker package to Docker's official repository, then installing the full Docker Engine toolchain including Docker Compose.

Observed Result: The apt update output confirms the new download.docker.com repository was fetched successfully alongside the existing Ubuntu repositories, and the installation of docker-ce and its related packages begins, indicating the repository setup was successful.

```
samikhan@osboxes:~$ sudo install -m 0755 -d /etc/apt/keyrings
samikhan@osboxes:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg
 \
-o /etc/apt/keyrings/docker.asc
samikhan@osboxes:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
samikhan@osboxes:~$ echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc
] https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" |
 \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
samikhan@osboxes:~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu plucky InRelease [32.5 kB]
Hit:2 http://archive.ubuntu.com/ubuntu plucky InRelease
Hit:3 http://security.ubuntu.com/ubuntu plucky-security InRelease
Get:4 https://download.docker.com/linux/ubuntu plucky/stable amd64 Packages [24.
7 kB]
Hit:5 http://archive.ubuntu.com/ubuntu plucky-updates InRelease
Hit:6 http://archive.ubuntu.com/ubuntu plucky-backports InRelease
Fetched 57.2 kB in 1s (69.2 kB/s)
All packages are up to date.
samikhan@osboxes:~$ sudo apt install -y docker-ce docker-ce-cli containerd.io do
cker-buildx-plugin docker-compose-plugin
Installing:
```

Figure 3: Adding Docker's official APT repository and installing Docker Engine.

Verifying the Docker Service Status

After installation, it is important to confirm that the Docker background service (the daemon that manages containers) has started automatically and is running correctly.

Command Used:

```
systemctl status docker.service
```

Command Explanation:

- systemctl status docker.service → displays whether the Docker service is loaded, enabled, and currently active, along with its most recent log entries.

What Is Being Done: The student is checking the health of the Docker daemon before attempting to use it, to rule out service-level problems before running any containers.

Observed Result: The output shows the service is "active (running)" and "enabled" (meaning it will also start automatically on boot), with the dockerd process bound to its process ID and its socket. This confirms Docker installed and started correctly.

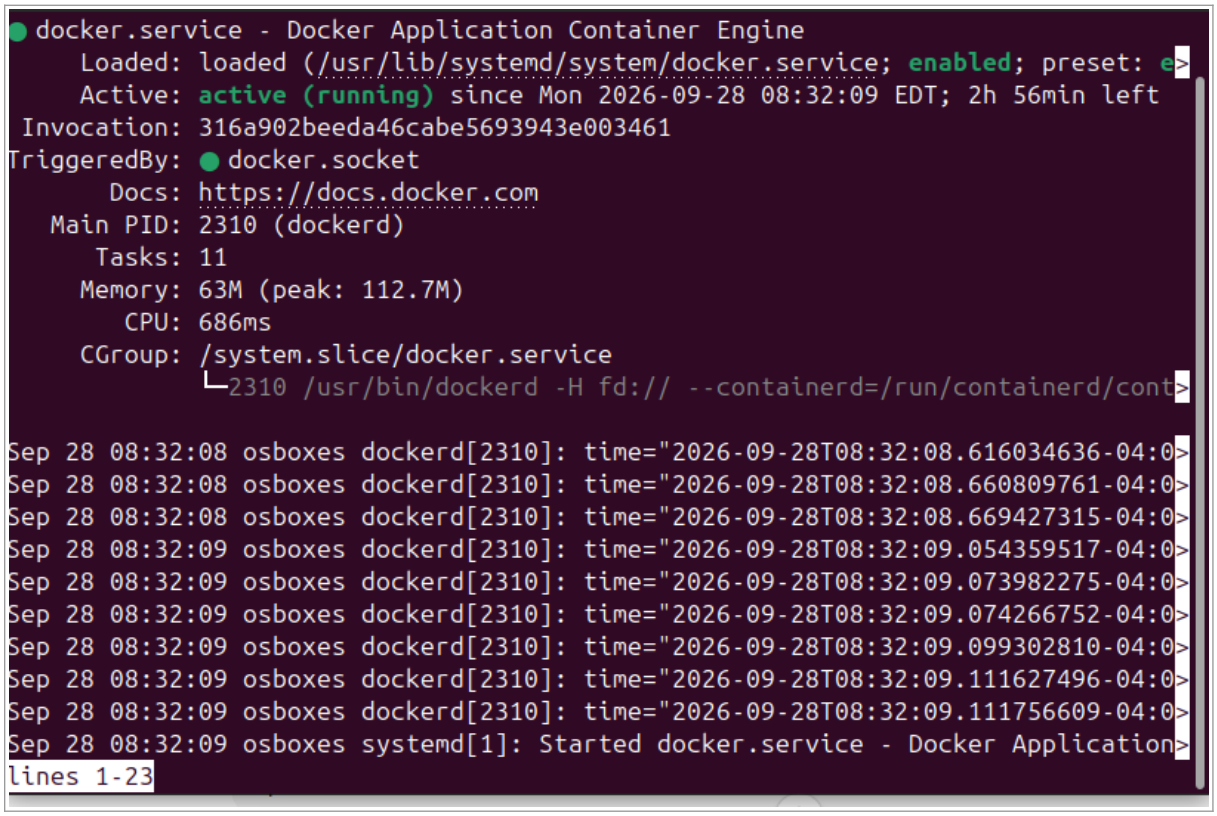


Figure 4: Verifying that the Docker service is active and running.

Testing the Docker Installation with hello-world

Docker provides a small official test image, hello-world, specifically designed to confirm that an installation is fully functional end-to-end.

Command Used:

```
sudo docker run hello-world
```

Command Explanation:

- docker run → creates and starts a new container from a specified image.
- hello-world → the name of the lightweight test image that Docker pulls from Docker Hub if it is not already available locally.

What Is Being Done: The student is performing Docker's recommended sanity check: pulling an image from Docker Hub, creating a container from it, running it, and printing its output back to the terminal.

Observed Result: The output shows the hello-world image being pulled successfully and the container printing its confirmation message, which explicitly states that the installation appears to be working correctly. This proves the Docker client can talk to the daemon, pull images from the registry, and run containers.

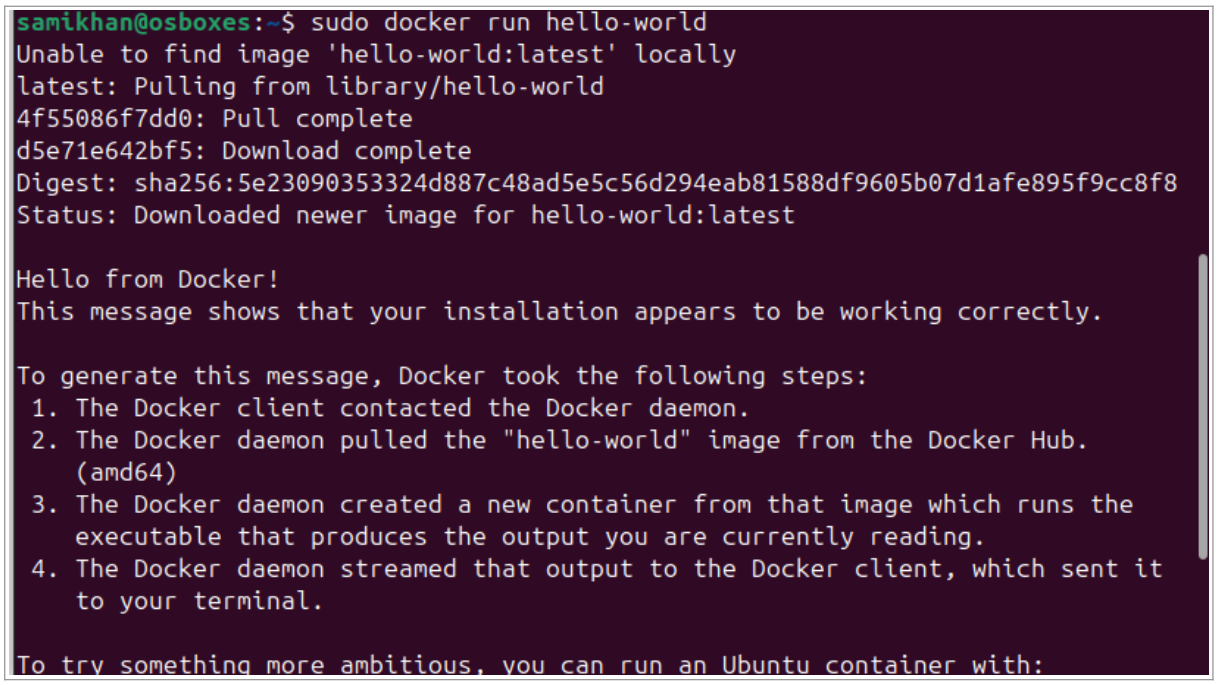


Figure 5: Running the hello-world test container to confirm Docker works end-to-end.

Confirming the Installed Docker and Compose Versions

With the installation verified, the exact installed versions of Docker Engine and Docker Compose are recorded for reference.

Command Used:

```
sudo docker --version
sudo docker compose version
```

Command Explanation:

- docker --version → prints the installed Docker Engine version and build number.
- docker compose version → prints the installed Docker Compose plugin version.

What Is Being Done: The student is documenting the exact software versions used for this lab, which is good practice for reproducibility.

Observed Result: The output confirms Docker version 29.2.1 (build a5c7197) and Docker Compose version v5.0.2 are installed.

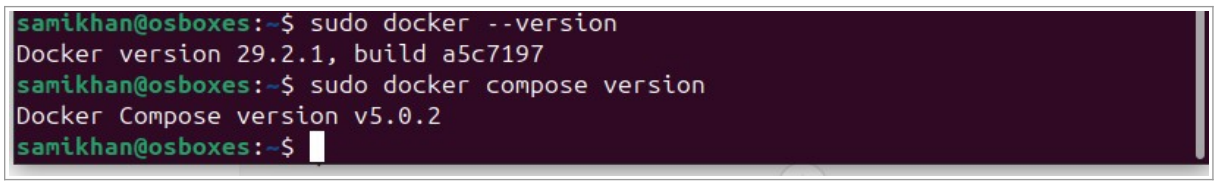


Figure 6: Confirming the installed Docker Engine and Docker Compose versions.

Enabling Docker Without sudo

By default, only the root user (or a user with sudo) can talk to the Docker daemon, because the docker.sock socket is owned by the root-level docker group. To run Docker commands conveniently without typing sudo every time, the current user is added to that group.

Command Used:

```
sudo usermod -aG docker $USER
newgrp docker
```

Command Explanation:

- usermod -aG docker $USER → appends (-a) the current user to the docker group (-G), granting permission to communicate with the Docker daemon socket.
- newgrp docker → starts a new shell session with the docker group applied immediately, without requiring a full logout and login.

What Is Being Done: The student is configuring the account so that Docker commands can be run directly by the samikhan user.

Observed Result: The commands complete without any errors, indicating the user was added to the docker group and the new group membership was applied to the current shell.

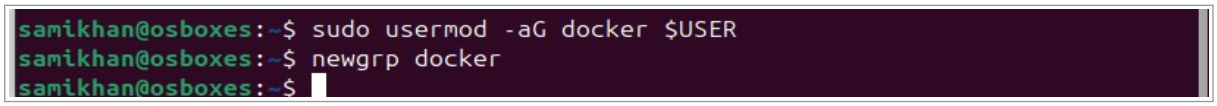


Figure 7: Adding the current user to the docker group to allow running Docker without sudo.

Verifying Containers and Images

As a final check before moving on to the containerization task, the current state of the Docker environment is inspected — which containers are running and which images are already downloaded.

Command Used:

```
docker ps
docker images
```

Command Explanation:

- docker ps → lists currently running containers.
- docker images → lists Docker images that are stored locally.

What Is Being Done: The student is confirming a clean starting state: no containers are left running from the hello-world test, and only the hello-world image is present locally.

Observed Result: docker ps returns an empty list (no running containers, since the hello-world container exits immediately after printing its message), and docker images shows the hello-world:latest image with its ID and disk usage. This confirms Docker is fully operational and ready for the containerization task.

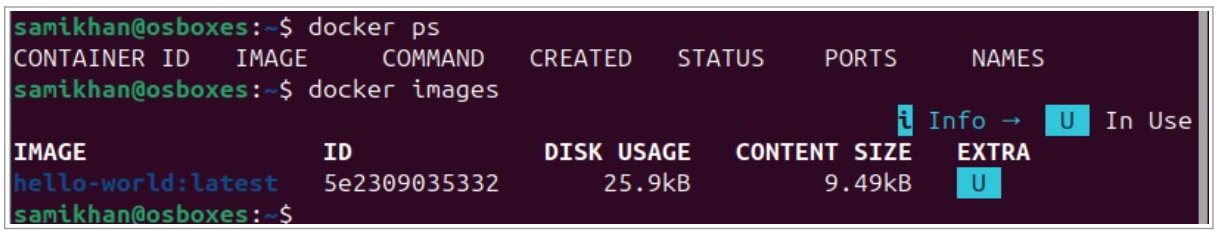


Figure 8: Verifying that no containers are running and that the hello-world image is present.

Part B: Containerizing a 3-Tier Laravel Application

With Docker confirmed to be working, the next stage of the lab is to build and run a three-tier "Student Management System" application: an Nginx frontend container, a PHP/Laravel backend container, and a MySQL database container, all defined together in a Docker Compose file inside the ~/laravel-3tier project directory.

Scaffolding the Laravel Project with Composer

Rather than installing PHP and Composer directly on the host machine, the student uses a throwaway Composer container to scaffold a new Laravel project. This keeps the host system clean and ensures the project is built using a consistent, containerized tool version.

Command Used:

```
docker run --rm \
 -v "$(pwd)/app" \
 -w /app \
 composer:2 \
 composer create-project laravel/laravel .
```

Command Explanation:

- docker run --rm → creates a temporary container that is automatically deleted once it finishes running.
- -v "$(pwd)/app" → mounts the current host directory into the container so that files created inside the container appear on the host.
- -w /app → sets /app as the working directory inside the container.
- composer:2 → the official Composer version 2 image, used only for this one command.
- composer create-project laravel/laravel . → downloads the Laravel framework and all of its PHP dependencies into the current directory.

What Is Being Done: The student is generating a brand-new Laravel application skeleton (routes, controllers, configuration, vendor dependencies) that will later be packaged into the backend Docker image.

Observed Result: Composer pulls its own image, then downloads dozens of Laravel dependency packages one by one, each reported as "Pull complete" or with a download progress bar.

```
samikhan@osboxes:~/laravel-3tier$ docker run --rm \
  -v "$(pwd):/app" \
  -w /app \
  composer:2 \
  composer create-project laravel/laravel .
Unable to find image 'composer:2' locally
2: Pulling from library/composer
406faa0b22ce: Pull complete
b643345218f3: Pull complete
92421a00585b: Pull complete
6361c0c16a9a: Pull complete
88c97ededfa5: Pull complete
0bfa53ef0584: Pull complete
108ad594590c: Pull complete
b7fa1c822b37: Pull complete
8806a571cc74: Pull complete
6f7b1962d764: Pull complete
52bc90a67a58: Pull complete
1755349a9517: Pull complete
e2de96513ba9: Pull complete
c74954296e3d: Pull complete
```

Figure 9: Pulling the composer:2 image and beginning the Laravel dependency download.

The dependency download continues, showing the remaining PHP packages (including PHPUnit and its supporting libraries) being fetched and extracted.

```
  - Downloading phar-io/manifest (2.0.4)
  - Downloading myclabs/deep-copy (1.14.0)
  - Downloading phpunit/phpunit (12.5.36)
   0/109 [>----------------------------]   0%
   3/109 [>----------------------------]   2%
  12/109 [===>------------------------]  11%
  21/109 [=====>----------------------]  19%
  25/109 [======>---------------------]  22%
  34/109 [========>-------------------]  31%
  44/109 [===========>----------------]  40%
  55/109 [==============>-------------]  50%
  63/109 [================>-----------]  57%
  68/109 [=================>----------]  62%
  78/109 [====================>-------]  71%
  86/109 [======================>-----]  78%
  88/109 [======================>-----]  80%
  99/109 [=========================>--]  90%
 105/109 [==========================>-]  96%
 107/109 [===========================>]  98%
 109/109 [============================] 100%
  - Installing doctrine/inflector (2.1.0): Extracting archive
```

Figure 10: Composer downloading and extracting the remaining Laravel dependencies.

Once all packages are discovered and installed, Composer automatically runs Laravel's post-install scripts, including publishing default asset configuration and generating the application's encryption key.

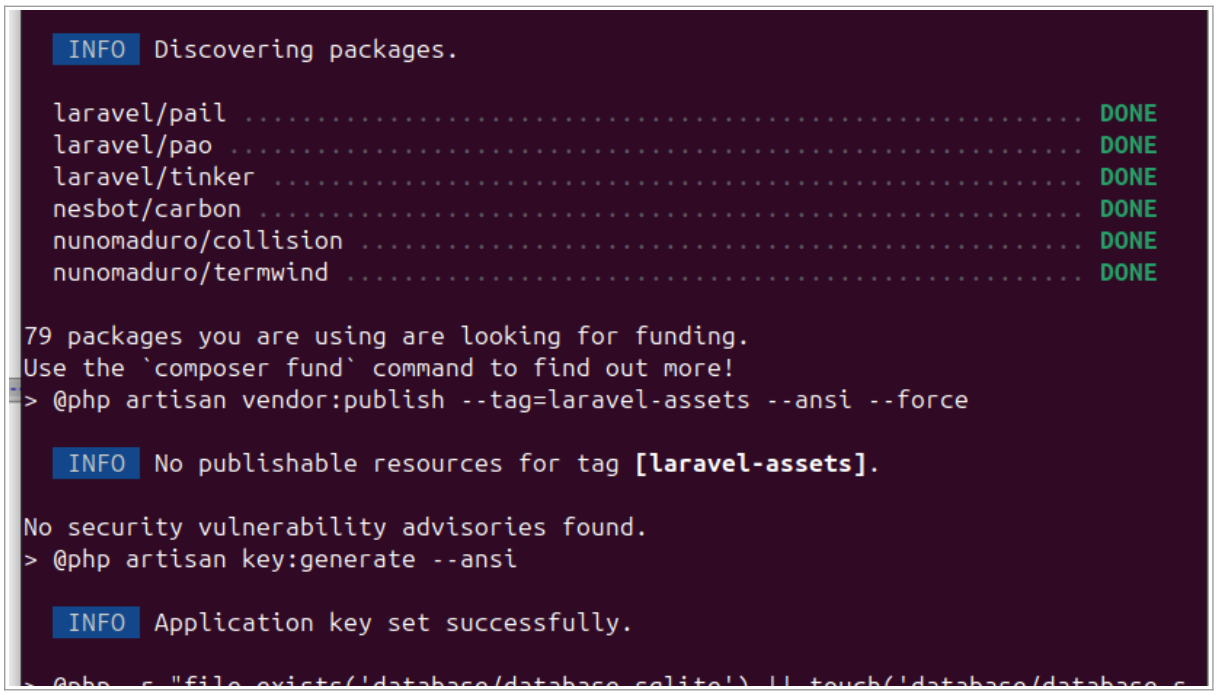


Figure 11: Composer finishing package discovery and Laravel's post-install scripts, including php artisan key:generate.

Finally, Composer creates the local SQLite database file used by Laravel's default setup and runs the initial framework migrations, creating the base users, cache, and jobs tables. This confirms the Laravel project was scaffolded successfully inside the laravel-3tier directory.

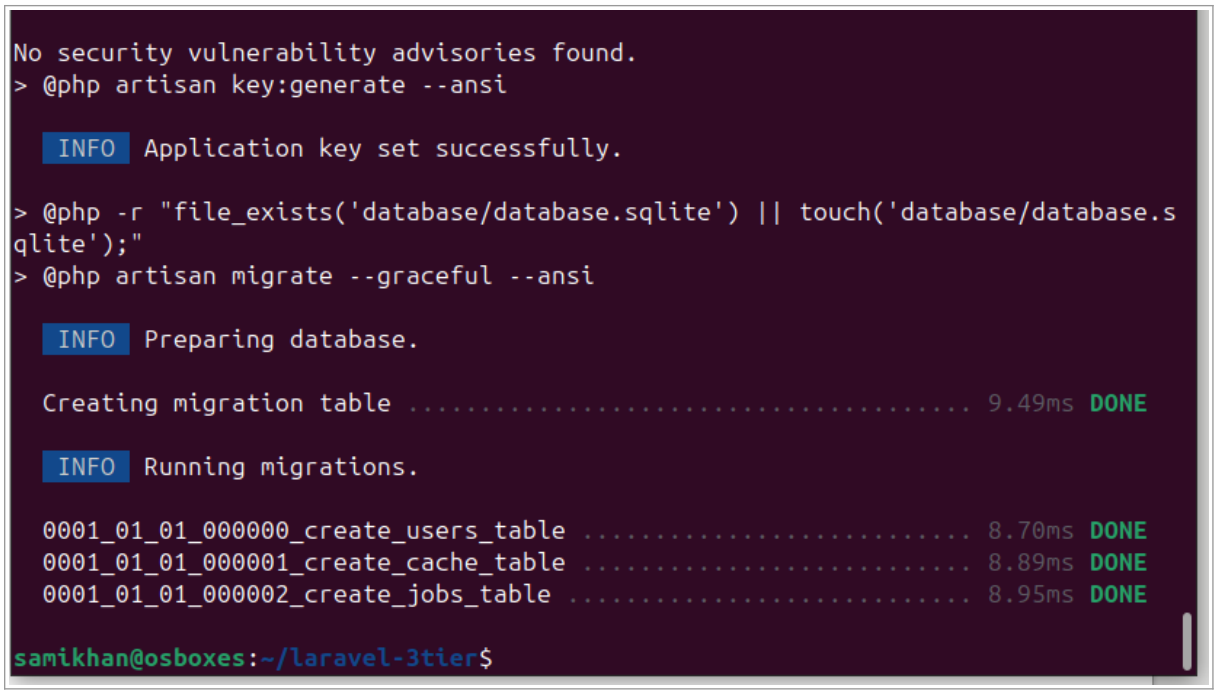


Figure 12: Laravel's initial database migrations completing successfully, confirming the project scaffold is ready.

Writing the Application Dockerfile

With the Laravel source code in place, a Dockerfile is written to describe how the backend PHP application should be packaged into a Docker image.

Command Used (Dockerfile contents, opened in nano):

```
FROM php:8.4-fpm

WORKDIR /var/www/html

RUN apt-get update && apt-get install -y \
 git \
 unzip \
 libzip-dev \
 libpng-dev \
 libonig-dev \
 libxml2-dev \
 && docker-php-ext-install \
 pdo_mysql \
 mbstring \
 bcmath \
 exif \
 pcntl \
 zip \
 && rm -rf /var/lib/apt/lists/*
```

Command Explanation:

- FROM php:8.4-fpm → uses the official PHP 8.4 FastCGI Process Manager image as the base for the backend container.
- WORKDIR /var/www/html → sets the standard web-root directory as the working directory for all subsequent instructions.
- RUN apt-get install … → installs the system libraries (zip, PNG, XML, onig) that Laravel's PHP extensions depend on.
- docker-php-ext-install pdo_mysql mbstring bcmath exif pcntl zip → compiles and enables the specific PHP extensions Laravel requires, most importantly pdo_mysql for talking to the MySQL database container.
- rm -rf /var/lib/apt/lists/* → removes the package index cache to keep the final image smaller.

What Is Being Done: The student is authoring the backend image definition in the nano editor, choosing a PHP-FPM base image and enabling exactly the extensions the Laravel application needs.

Observed Result: The nano editor shows the Dockerfile content as written, confirming the base image and required PHP extensions are correctly specified before the file is saved.

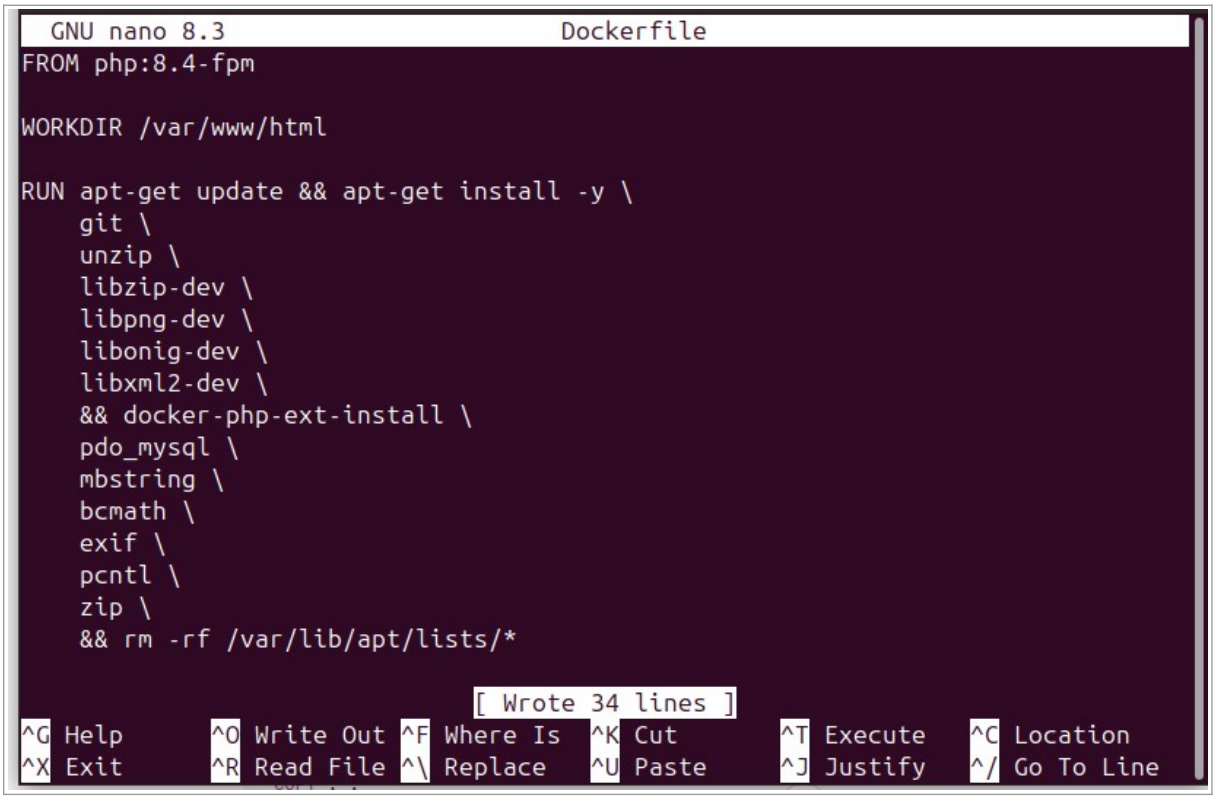


Figure 13: Writing the backend Dockerfile in nano, based on the php:8.4-fpm image.

Before finishing the Dockerfile, the student returns to the project directory to confirm the Laravel version that was installed and to prepare the folder that will hold the Nginx configuration.

Command Used:

```
cd ~/laravel-3tier
grep '"laravel/framework"' composer.json
mkdir -p nginx
ls
nano Dockerfile
```

Command Explanation:

- cd ~/laravel-3tier → moves into the Laravel project directory created earlier.
- grep '"laravel/framework"' composer.json → searches the Composer manifest for the exact Laravel framework version that was installed.
- mkdir -p nginx → creates a subdirectory to hold the Nginx-specific configuration files used by the frontend container.
- ls → lists the project files to confirm the standard Laravel folder structure (app, routes, database, public, etc.) is present.
- nano Dockerfile → opens the backend Dockerfile for editing.

What Is Being Done: The student is confirming the exact Laravel version in use and organising the project directory (creating the nginx/ folder) before continuing to build out the Docker configuration files.

Observed Result: The grep command confirms laravel/framework version "^13.17" is installed, and the directory listing shows a complete, standard Laravel project structure, confirming the scaffold from the previous step was successful.

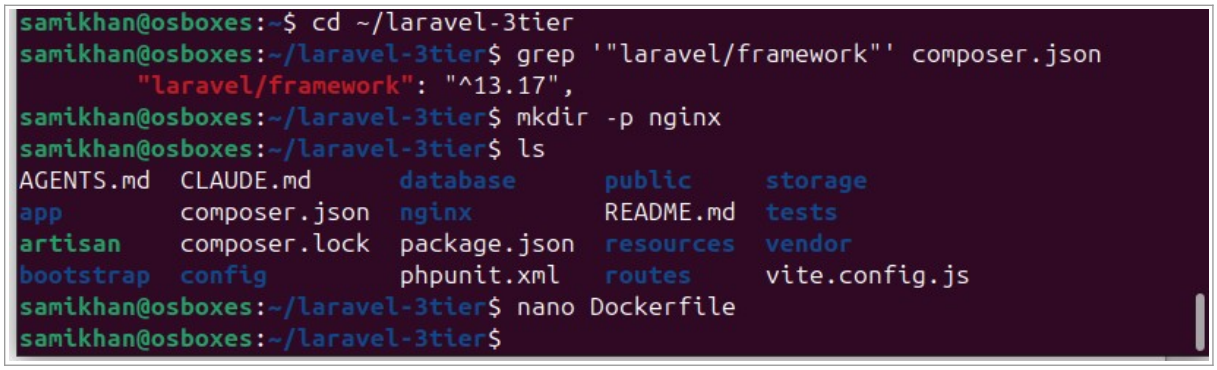


Figure 14: Confirming the installed Laravel framework version and preparing the nginx directory.

Creating the Nginx Configuration and Docker Compose Files

To complete the three-tier setup, the backend Dockerfile is finished, and Nginx configuration, a Docker Compose file, and an environment file are created to tie the frontend, backend, and database services together.

Command Used:

```
RUN chown -R www-data:www-data storage bootstrap/cache
EXPOSE 9000
CMD ["php-fpm"]

ls -l Dockerfile
mkdir -p nginx
nano nginx/default.conf
nano nginx/Dockerfile
ls -l nginx
nano compose.yaml
nano .env
docker compose up -d --build
```

Command Explanation:

- chown -R www-data:www-data storage bootstrap/cache → gives the web server user ownership of Laravel's writable storage and cache directories, which is required for the application to run correctly.
- EXPOSE 9000 → documents that the PHP-FPM process listens on port 9000 inside the container.
- CMD ["php-fpm"] → sets the default command that starts the PHP-FPM process when the backend container launches.
- nano nginx/default.conf and nano nginx/Dockerfile → create the Nginx site configuration and a small Dockerfile that builds the frontend image (based on nginx) and copies that configuration into it.
- nano compose.yaml → defines the three services (frontend, backend, db), their images, ports, volumes, and the shared network that lets them communicate.
- nano .env → stores environment variables (such as database credentials) used by Laravel and referenced by Compose.
- docker compose up -d --build → builds any images defined in compose.yaml that do not already exist, then starts all defined services in detached (-d) mode.

What Is Being Done: The student is finishing the backend Dockerfile and assembling every remaining piece — the Nginx reverse-proxy configuration, the Docker Compose file that wires the three services together, and the environment file — before launching the full stack for the first time.

Observed Result: The directory listings confirm the Dockerfile and the new nginx/ files (default.conf and Dockerfile) were created with the expected sizes. The final docker compose up -d --build command begins executing and starts pulling the mysql:8.0 image for the database service.

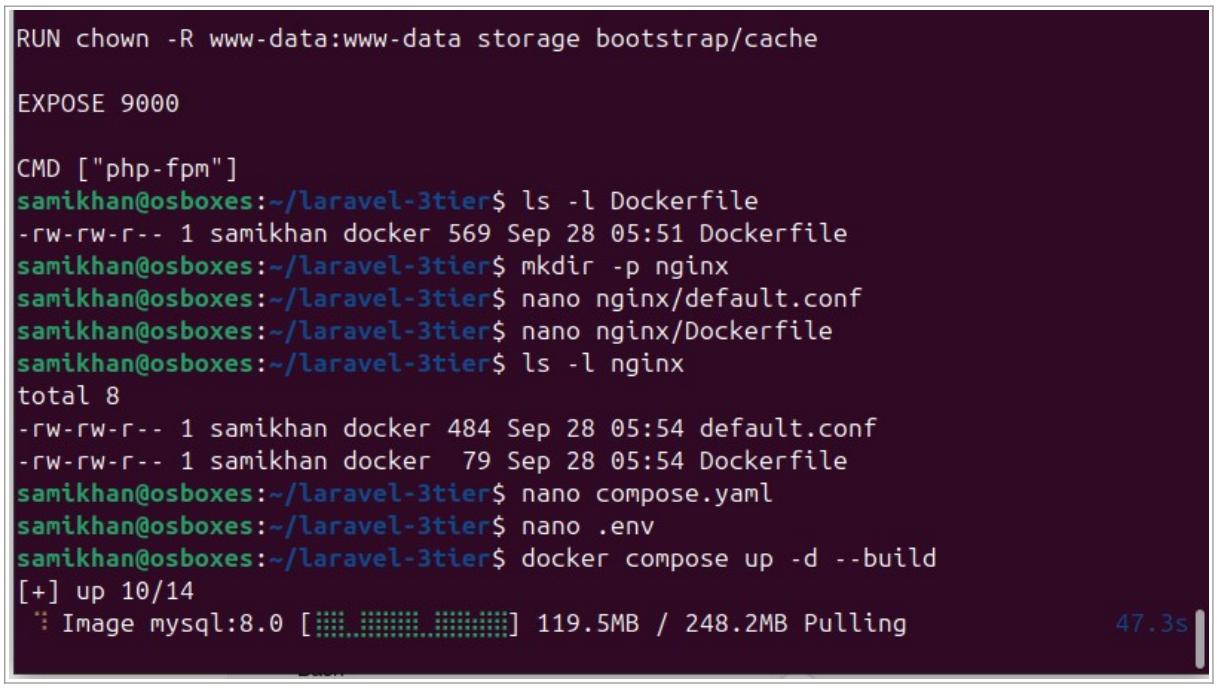


Figure 15: Finishing the backend Dockerfile, creating the Nginx and Compose configuration files, and starting the first build.

Resolving a Docker Permission Error

On the first attempt to rebuild and start the stack, Docker returns a permission error rather than building the images.

Command Used:

```
docker compose down
docker compose build
docker compose up -d
```

Command Explanation:

- docker compose down → stops and removes any containers, networks defined for the project.
- docker compose build → (re)builds the images for all services defined in compose.yaml.
- docker compose up -d → starts the services in the background.

What Is Being Done: The student attempts to rebuild the stack, but the shell used for this attempt has not yet picked up the docker group membership that was configured earlier with newgrp docker — in a new terminal/session, that group change had not taken effect.

Observed Result (Error): Every command fails with “permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock”, because this shell session is not yet part of the docker group. The student responds by re-running the group assignment command, sudo usermod -aG docker $USER, to fix the permission on this session.

```
samikhan@osboxes:~/laravel-3tier$ docker compose down
docker compose build
docker compose up -d
permission denied while trying to connect to the Docker daemon socket at unix://
/var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.51/containers/js
on?all=1&filters=%7B%22label%22%3A%7B%22com.docker.compose.config-hash%22%3Atrue
%2C%22com.docker.compose.oneoff%3DFalse%22%3Atrue%2C%22com.docker.compose.projec
t%3Dlaravel-3tier%22%3Atrue%7D%7D": dial unix /var/run/docker.sock: connect: per
mission denied
[+] Building 0.0s (0/0)
[+] build 0/2
 ⠿ Image laravel-3tier-frontend Building                                    0.1s
 ⠿ Image laravel-3tier-backend  Building                                    0.1s
permission denied while trying to connect to the docker API at unix:///var/run/d
ocker.sock

unable to get image 'laravel-3tier-frontend': permission denied while trying to
connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://
%2Fvar%2Frun%2Fdocker.sock/v1.51/images/laravel-3tier-frontend/json": dial unix
/var/run/docker.sock: connect: permission denied
samikhan@osboxes:~/laravel-3tier$ sudo usermod -aG docker $USER
[sudo] password for samikhan:
```

Figure 16: A Docker daemon permission error caused by the current shell not having docker-group access.

Building the Backend and Frontend Images

Since the group membership change had not yet propagated to the current shell, the student proceeds by prefixing the Docker commands with sudo, which immediately restores access to the Docker daemon and allows the build to continue.

Command Used:

```
sudo docker ps
sudo docker compose down
sudo docker compose build
sudo docker compose up -d
```

Command Explanation:

- sudo docker ps → confirms Docker access works when run with administrator privileges.
- sudo docker compose build → builds the frontend and backend images defined in compose.yaml.
- sudo docker compose up -d → starts all services once the images are built.

What Is Being Done: The student is building both custom images defined in the project: laravel-3tier-frontend (based on nginx:alpine, with the custom Nginx configuration copied in) and laravel-3tier-backend (based on php:8.4-fpm, with Composer used to install PHP dependencies inside the image).

Observed Result: sudo docker ps succeeds and returns an empty container list (no permission error this time), and the build output shows Docker pulling the nginx:alpine and php:8.4-fpm base images and beginning the multi-stage build for both the frontend and backend services.

```
ar/run/docker.sock: connect: permission denied
samikhan@osboxes:~/laravel-3tier$ sudo docker ps
[sudo] password for samikhan:
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
samikhan@osboxes:~/laravel-3tier$ sudo docker compose down
sudo docker compose build
sudo docker compose up -d
[+] Building 21.2s (14/21)
 => [frontend 1/2] FROM docker.io/library/nginx:alpine@sha256:df221db836e  4.0s
 => => resolve docker.io/library/nginx:alpine@sha256:df221db836e175408919  0.0s
 => => sha256:e72112c14215406dc7f23ef8e5cfdfddeb8662176 20.53MB / 20.53MB  3.7s
 => => extracting sha256:e72112c14215406dc7f23ef8e5cfdfddeb8662176167382a  0.3s
 => [backend stage-0 1/7] FROM docker.io/library/php:8.4-fpm@sha256:b0d7  15.6s
 => => resolve docker.io/library/php:8.4-fpm@sha256:b0d7dff8f2d57155aebb5  0.0s
 => => sha256:167e34ed0b9bb382f79fe246b868a5ba5d50a89 34.60MB / 117.84MB  15.6s
 => => sha256:6b37362b3da78869050b894b799ad4df04f1f3b5 16.78MB / 29.83MB  15.6s
 => CACHED [backend] FROM docker.io/library/composer:2@sha256:9715c7f6904  0.0s
 => => resolve docker.io/library/composer:2@sha256:9715c7f69044da2a212a5f  0.0s
 => [backend internal] load build context                                  1.7s
 => => transferring context: 49.95MB                                       1.6s
 => [frontend 2/2] COPY nginx/default.conf /etc/nginx/conf.d/default.conf  0.1s
 => [frontend] exporting to image                                          0.1s
```

Figure 17: Docker access restored with sudo; the frontend and backend images begin building.

The build continues: the backend stage installs Composer and its dependencies while the frontend stage finishes copying the custom Nginx configuration and exporting its image layers.

```
 => => extracting sha256:e72112c14215406dc7f23ef8e5cfdfddeb8662176167382a  0.3s
 => [backend stage-0 1/7] FROM docker.io/library/php:8.4-fpm@sha256:b0d7  33.4s
 => => resolve docker.io/library/php:8.4-fpm@sha256:b0d7dff8f2d57155aebb5  0.0s
 => => sha256:167e34ed0b9bb382f79fe246b868a5ba5d50a89 65.01MB / 117.84MB  33.3s
 => => sha256:6b37362b3da78869050b894b799ad4df04f1f3b5 29.83MB / 29.83MB  22.2s
 => => extracting sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774087d  0.5s
 => => extracting sha256:acb93b1ad7af1b2613c52d852097bd1a620edb8287b639e5  0.0s
 => CACHED [backend] FROM docker.io/library/composer:2@sha256:9715c7f6904  0.0s
 => => resolve docker.io/library/composer:2@sha256:9715c7f69044da2a212a5f  0.0s
 => [backend internal] load build context                                  1.7s
 => => transferring context: 49.95MB                                       1.6s
 => [frontend 2/2] COPY nginx/default.conf /etc/nginx/conf.d/default.conf  0.1s
 => [frontend] exporting to image                                          0.1s
 => => exporting layers                                                    0.0s
 => => exporting manifest sha256:0314a33f1d1f214bed0c7cda194036ad067aed21  0.0s
 => => exporting config sha256:5ef0dd525f990ac67e43f209fa0b1242460f22f462  0.0s
 => => exporting attestation manifest sha256:74fe5586f3997abca177200deb4c  0.0s
 => => exporting manifest list sha256:856124b04166ab79c36768d3143ec37d4d9  0.0s
 => => naming to docker.io/library/laravel-3tier-frontend:latest           0.0s
 => => unpacking to docker.io/library/laravel-3tier-frontend:latest        0.0s
 => [frontend] resolving provenance for metadata file                      0.0s
```

Figure 18: The backend and frontend build stages progressing, with the frontend image being exported and tagged.

Completing the Build and Starting the Containers

What Is Being Done: The build process finishes installing the backend's Composer dependencies, sets file ownership with chown, and exports both final images.

Observed Result: Both images report as successfully Built (laravel-3tier-backend and laravel-3tier-

frontend), and docker compose up then creates the shared laravel-3tier_laravel_network, two named volumes for persistent data (laravel-3tier_backend_data and laravel-3tier_db_data), and the three containers: laravel-db, laravel-backend, and laravel-frontend. This confirms the entire stack was built and started successfully.

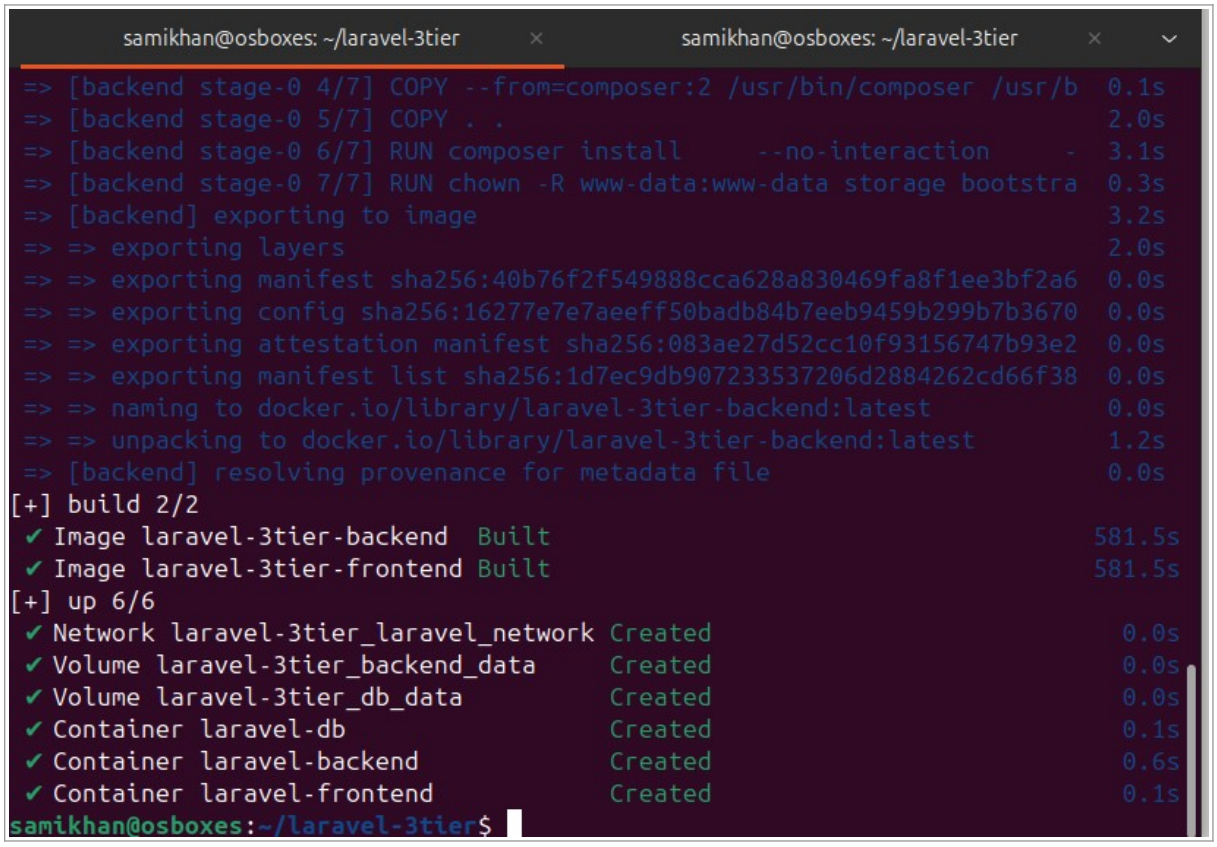


Figure 19: Both images built successfully and all three containers (db, backend, frontend) created.

Verifying the Running Containers

With the stack started, the running services are checked to confirm all three containers are healthy and their ports are mapped as expected.

Command Used:

```
sudo docker compose ps
```

Command Explanation:

- docker compose ps → lists the status, image, and port mappings of every container belonging to the current Compose project.

What Is Being Done: The student is verifying that the frontend, backend, and database containers are all in the "Up" state before testing the application in a browser.

Observed Result: All three containers show status "Up 5 minutes": laravel-backend (laravel-3tier-backend image, port 9000/tcp), laravel-db (mysql:8.0 image, port 3306/tcp), and laravel-frontend (laravel-3tier-frontend image, mapped from host port 8080 to container port 80). This confirms the stack is fully running.

```
samikhan@osboxes:~/laravel-3tier$ sudo docker compose ps
NAME               IMAGE                    COMMAND                  SERVICE
CREATED         STATUS         PORTS
laravel-backend    laravel-3tier-backend    "docker-php-entrypoi…"   backend
5 minutes ago   Up 5 minutes   9000/tcp
laravel-db         mysql:8.0                "docker-entrypoint.s…"   db
5 minutes ago   Up 5 minutes   3306/tcp, 33060/tcp
laravel-frontend   laravel-3tier-frontend   "/docker-entrypoint.…"   frontend
5 minutes ago   Up 5 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
samikhan@osboxes:~/laravel-3tier$
```

Figure 20: Confirming all three containers are running via docker compose ps.

Testing the Application in the Browser

With the frontend container mapped to host port 8080, the application is opened in a web browser to confirm the full stack — Nginx, PHP-FPM, and MySQL — is working together correctly.

What Is Being Done: The student navigates to http://localhost:8080, which is served by the Nginx frontend container and proxied through to the Laravel backend.

Observed Result: The "Student Management System" page loads successfully, with a form for adding students and a table listing existing records — initially showing one pre-existing record, "Sami Khan." This confirms the Nginx-to-PHP-FPM connection and the database connection are both working.

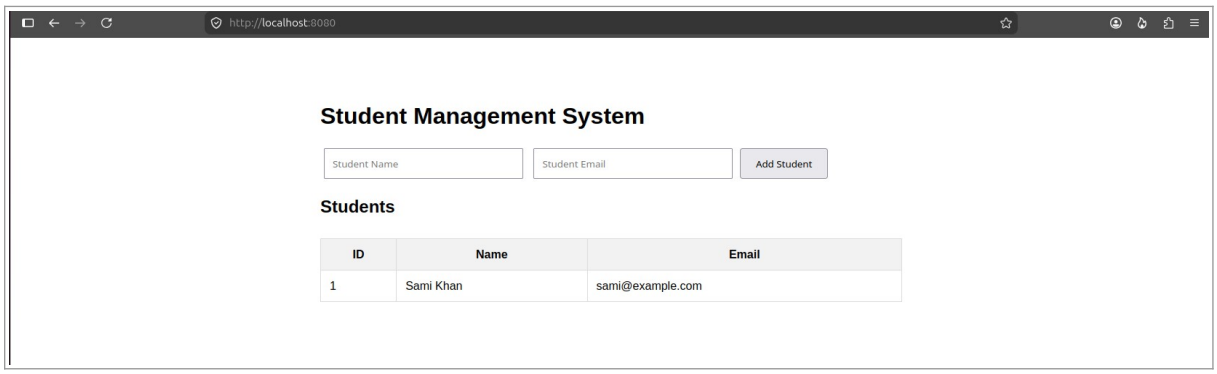


Figure 21: The Student Management System application loaded successfully at http://localhost:8080.

To confirm the application can also write new data, a second student record is added through the form.

Observed Result: The students table now shows two rows, with the newly added record ("Sami khan", khansami1800@gmail.com) appearing alongside the original one. This proves that form submissions from the browser are correctly reaching the backend container and being written to the MySQL database container.

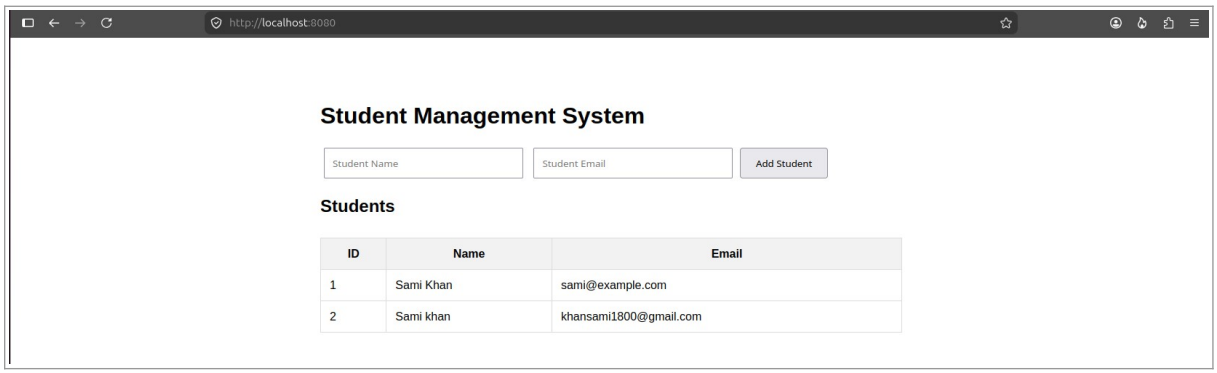


Figure 22: A new student record successfully added, confirming write access to the database.

Part C: Verifying Internal Container Communication

The final part of the lab demonstrates that the three containers form a genuine multi-tier system rather than three isolated processes: application code is deployed directly into the running backend container, and the internal Docker network, DNS resolution, and volume persistence between the backend and database containers are all verified.

Deploying the Student Management Feature into the Backend Container

The Student Management feature (model, controller, view, and route) is added to the project on the host and then copied directly into the already-running backend container using docker cp, alongside a new database migration for the students table.

Command Used:

```
cd ~/laravel-3tier
nano database/migrations/2026_09_28_000000_create_students_table.php
sudo docker cp app/Models/Student.php \
 laravel-backend:/var/www/html/app/Models/Student.php
sudo docker cp app/Http/Controllers/StudentController.php \
 laravel-backend:/var/www/html/app/Http/Controllers/StudentController.php
sudo docker cp resources/views/students.blade.php \
 laravel-backend:/var/www/html/resources/views/students.blade.php
sudo docker cp routes/web.php \
 laravel-backend:/var/www/html/routes/web.php
sudo docker cp database/migrations/2026_09_28_000000_create_students_table.php \
 laravel-backend:/var/www/html/database/migrations/2026_09_28_000000_create_students_table.php
```

Command Explanation:

- nano database/migrations/…create_students_table.php → writes a new Laravel migration file that defines the structure of the students table.
- docker cp <host path> <container>:<container path> → copies a file from the host filesystem directly into a running container's filesystem, without rebuilding the image.
- Each docker cp command places one piece of the feature (the Student model, its controller, its Blade view, the route definition, and the migration) into the matching location inside the laravel-backend container.

What Is Being Done: The student is deploying new application code straight into the live backend container, which is a quick way to iterate during development without rebuilding the Docker image

each time.

Observed Result: Each docker cp command reports “Successfully copied” along with the file size, confirming the Student model, controller, Blade view, routes file, and migration were all placed correctly inside the laravel-backend container.

```
samikhan@osboxes:~$ cd ~/laravel-3tier
samikhan@osboxes:~/laravel-3tier$ nano database/migrations/2026_09_28_000000_cre
ate_students_table.php
samikhan@osboxes:~/laravel-3tier$ sudo docker cp app/Models/Student.php laravel-
backend:/var/www/html/app/Models/Student.php
[sudo] password for samikhan:
Successfully copied 2.05kB to laravel-backend:/var/www/html/app/Models/Student.p
hp
samikhan@osboxes:~/laravel-3tier$ sudo docker cp app/Http/Controllers/StudentCon
troller.php laravel-backend:/var/www/html/app/Http/Controllers/StudentController
.php
Successfully copied 2.56kB to laravel-backend:/var/www/html/app/Http/Controllers
/StudentController.php
samikhan@osboxes:~/laravel-3tier$ sudo docker cp resources/views/students.blade.
php laravel-backend:/var/www/html/resources/views/students.blade.php
Successfully copied 3.07kB to laravel-backend:/var/www/html/resources/views/stud
ents.blade.php
samikhan@osboxes:~/laravel-3tier$ sudo docker cp routes/web.php laravel-backend:
/var/www/html/routes/web.php
Successfully copied 2.05kB to laravel-backend:/var/www/html/routes/web.php
samikhan@osboxes:~/laravel-3tier$ sudo docker cp database/migrations/2026_09_28_
000000_create_students_table.php laravel-backend:/var/www/html/database/migratio
ns/2026_09_28_000000_create_students_table.php
Successfully copied 2.56kB to laravel-backend:/var/www/html/database/migrations/
```

Figure 23: Copying the Student model, controller, view, and routes into the running backend container using docker cp.

Once the files are in place, the new migration is run inside the container and the Student model is exercised directly through Laravel’s interactive shell, Tinker.

Command Used:

```
sudo docker exec laravel-backend php artisan migrate
sudo docker exec laravel-backend php artisan tinker \
 --execute="App\\Models\\Student::create(['name'=>'Sami Khan','email'=>'sami@example.com']);"
sudo docker exec laravel-backend php artisan tinker \
 --execute="App\\Models\\Student::all();"
```

Command Explanation:

- docker exec laravel-backend <command> → runs a command inside the already-running laravel-backend container.
- php artisan migrate → applies any pending database migrations, creating the students table defined in the new migration file.
- php artisan tinker --execute="…" → runs a single line of PHP inside Laravel’s Tinker REPL; here it is used first to create a Student record and then to fetch all Student records.

What Is Being Done: The student is creating the students table in the MySQL container and then verifying, directly from the command line, that the backend container can both write to and read from that table through Laravel’s Eloquent ORM.

Observed Result: php artisan migrate reports the 2026_09_28_000000_create_students_table migration as DONE, confirming the table was created. The subsequent Tinker commands execute without error, confirming the backend container successfully reached the database container over the network to create and query the new record.

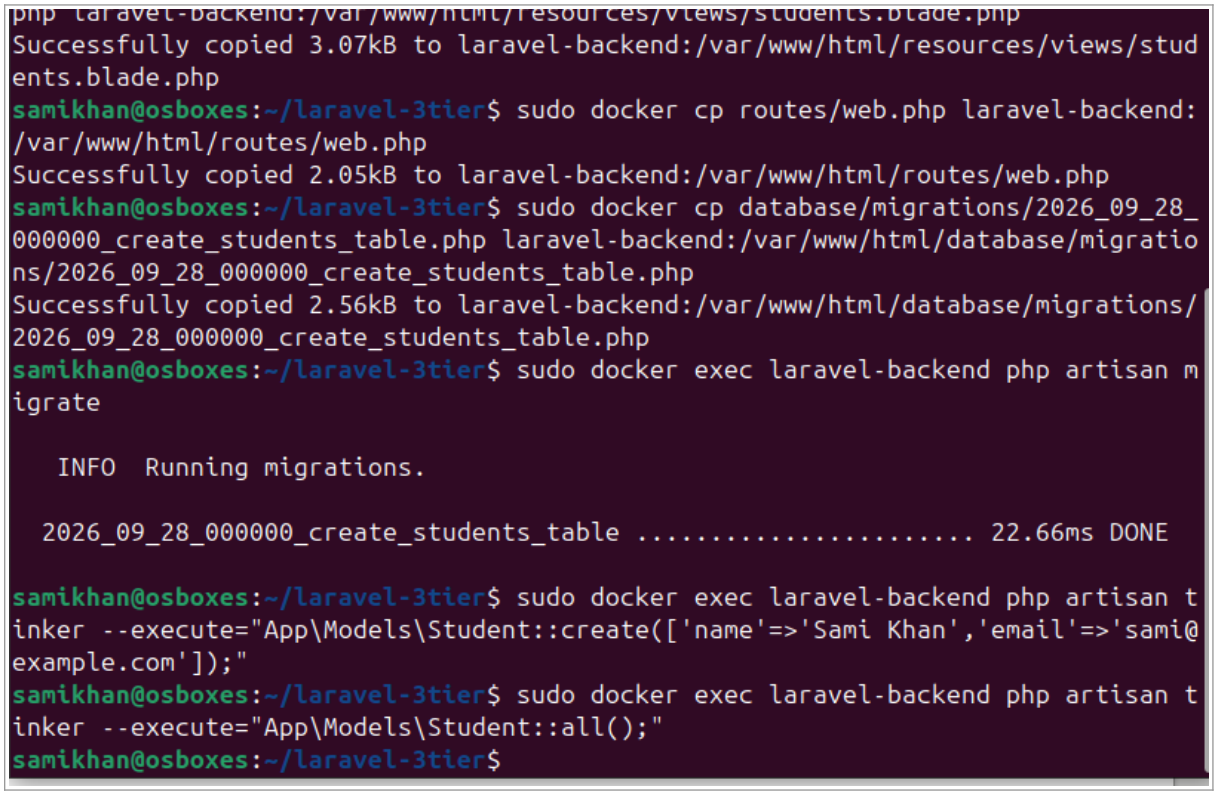


Figure 24: Running the students table migration and testing the Student model with php artisan tinker.

Inspecting the Docker Network

To understand how the three containers can reach each other, the Docker network that Compose created automatically is inspected.

Command Used:

```
sudo docker network inspect laravel-3tier_laravel_network
```

Command Explanation:

- docker network inspect <name> → prints detailed configuration for a Docker network, including its driver, subnet, gateway, and connected containers.

What Is Being Done: The student is examining the private bridge network that Docker Compose automatically created for this project, which is what allows the frontend, backend, and database containers to communicate with each other while staying isolated from other Docker projects on the host.

Observed Result: The output shows the network laravel-3tier_laravel_network is a bridge-type network with IPv4 enabled, using the subnet 172.18.0.0/16 and gateway 172.18.0.1. This is the private address range from which each container in the project receives its internal IP address.

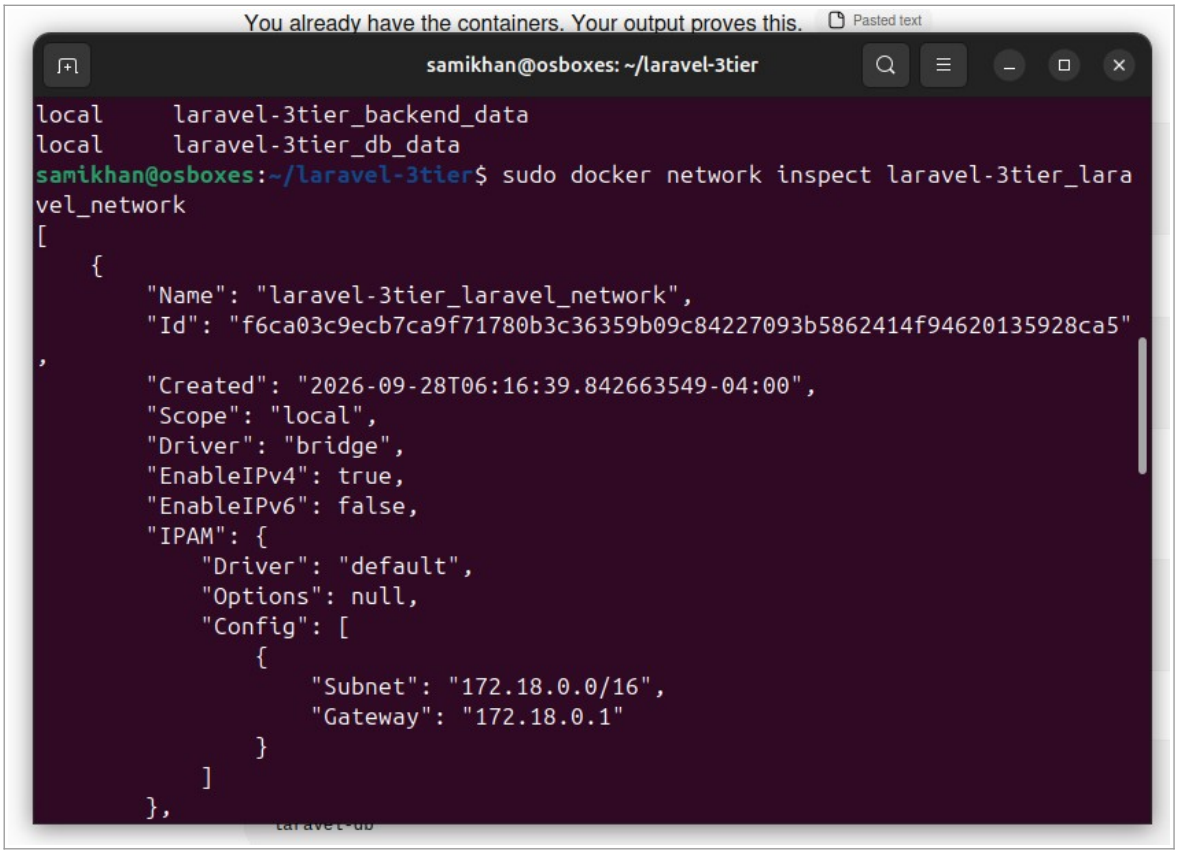


Figure 25: Inspecting the Docker Compose bridge network and its subnet configuration.

Verifying DNS Resolution Between Containers

Docker Compose automatically provides DNS resolution between containers on the same network, using each service's name from compose.yaml as its hostname. This is tested directly from inside the backend container.

Command Used:

```
sudo docker exec laravel-backend php -r "echo gethostbyname('db');"
```

Command Explanation:

- docker exec laravel-backend php -r "…" → runs an inline PHP snippet inside the backend container.
- gethostbyname('db') → a PHP function that resolves a hostname to an IP address; here, 'db' is the service name given to the MySQL container in compose.yaml.

What Is Being Done: The student is proving that the backend container can find the database container purely by its Compose service name, db, without needing to know or hard-code its IP address.

Observed Result: The command returns 172.18.0.2, the internal IP address that Docker assigned to the laravel-db container on the laravel-3tier_laravel_network. This confirms Docker's built-in DNS correctly resolves the service name db to the database container's address, which is exactly the mechanism the Laravel backend relies on to connect to MySQL using DB_HOST=db in its configuration.

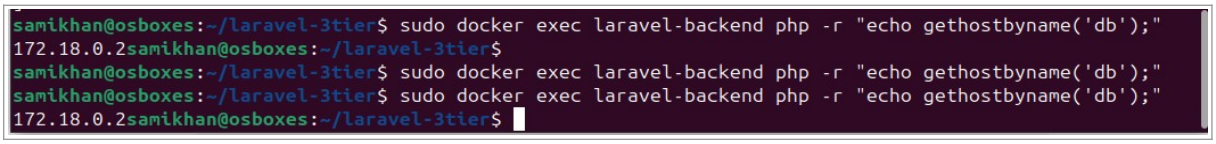


Figure 26: Resolving the hostname 'db' from inside the backend container to confirm internal DNS.

Testing Data Persistence with Docker Volumes

The final check demonstrates one of the key benefits of Docker volumes: data stored by the database container survives even if the container itself is deleted and recreated. A small test table is created directly inside the running MySQL container to set up this test.

Command Used:

```
sudo docker exec laravel-backend php -r "echo gethostbyname('db');"
sudo docker exec laravel-db mysql -ularavel -ppassword laravel -e \
 "CREATE TABLE IF NOT EXISTS docker_test (id INT AUTO_INCREMENT PRIMARY KEY, \
 message VARCHAR(255)); INSERT INTO docker_test (message) \
 VALUES ('Volume Persistence Test'); SELECT * FROM docker_test;"
```

Command Explanation:

- docker exec laravel-db mysql -ularavel -ppassword laravel -e "…" → connects to the MySQL server inside the laravel-db container as the laravel user and runs one or more SQL statements directly against the laravel database.
- CREATE TABLE IF NOT EXISTS docker_test (…) → creates a small table with an auto-incrementing id and a text message column.
- INSERT INTO docker_test (message) VALUES ('Volume Persistence Test') → inserts one test row.
- SELECT * FROM docker_test → reads back the row that was just inserted.

What Is Being Done: The student re-confirms the DNS check and then creates a small, disposable table with a single row of test data directly inside the MySQL container, specifically so that its survival can be checked after the container is destroyed and rebuilt.

Observed Result: The table is created, the row is inserted, and the SELECT statement confirms the row (id 1, message "Volume Persistence Test") exists inside the database container.

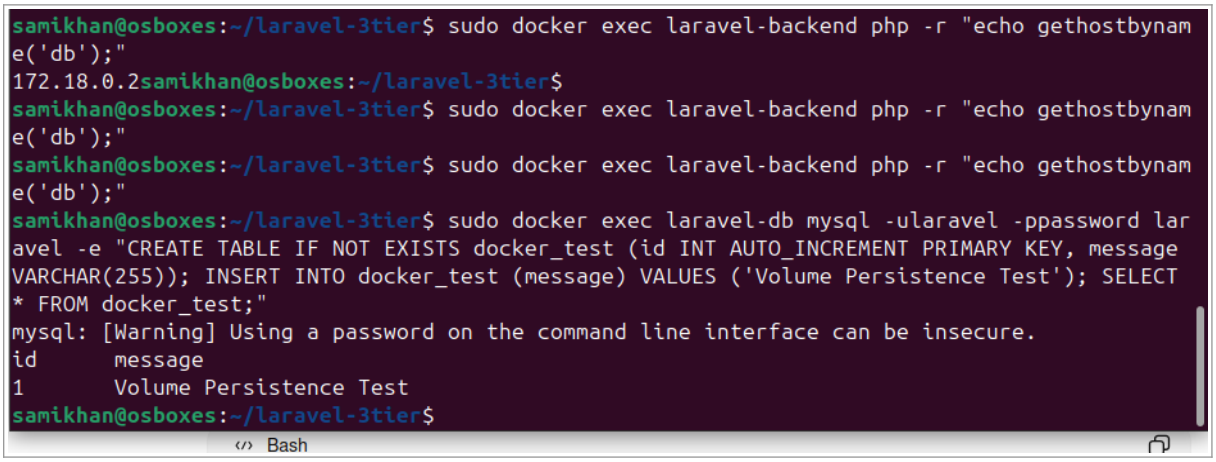


Figure 27: Creating a test table and row inside the laravel-db container ahead of the persistence test.

The database container is then deliberately removed and recreated, to prove that the test data is stored

in the named Docker volume rather than in the container itself.

Command Used:

```
sudo docker rm -f laravel-db
sudo docker compose up -d db
```

Command Explanation:

- docker rm -f laravel-db → forcefully stops and permanently deletes the laravel-db container.
- docker compose up -d db → recreates only the db service from compose.yaml as a brand new container, re-attaching it to the same named volume defined for that service.

What Is Being Done: The student intentionally destroys the database container, which would normally be expected to wipe out any data stored only inside that container, then immediately recreates it from the same Compose definition.

Observed Result: The old container is removed successfully, and Compose reports a fresh laravel-db container as Created, ready to be tested.

```
samikhan@osboxes:~/laravel-3tier$ sudo docker exec laravel-backend php -r "echo gethostbynam
e('db');"
samikhan@osboxes:~/laravel-3tier$ sudo docker exec laravel-db mysql -ularavel -ppassword lar
avel -e "CREATE TABLE IF NOT EXISTS docker_test (id INT AUTO_INCREMENT PRIMARY KEY, message
VARCHAR(255)); INSERT INTO docker_test (message) VALUES ('Volume Persistence Test'); SELECT
* FROM docker_test;"
mysql: [Warning] Using a password on the command line interface can be insecure.
id      message
1       Volume Persistence Test
samikhan@osboxes:~/laravel-3tier$ sudo docker rm -f laravel-db
laravel-db
samikhan@osboxes:~/laravel-3tier$ sudo docker compose up -d db
[+] up 1/1
 ✔ Container laravel-db Created                                                    0.1s
samikhan@osboxes:~/laravel-3tier$
```

Figure 28: Removing the laravel-db container and recreating it from the same Compose service definition.

Finally, the test table is queried again on the newly recreated container to check whether the data survived.

Command Used:

```
sudo docker exec laravel-db mysql -ularavel -ppassword laravel -e \
 "SELECT * FROM docker_test;"
```

What Is Being Done: The student queries the docker_test table on the brand-new container instance to see whether the row inserted before the container was deleted is still present.

Observed Result: The SELECT statement returns the same row (id 1, message “Volume Persistence Test”) even though the container that originally created it no longer exists. This confirms that the laravel-3tier_db_data named volume, not the container filesystem, is what actually stores the MySQL data — exactly the behaviour Docker volumes are designed to provide.

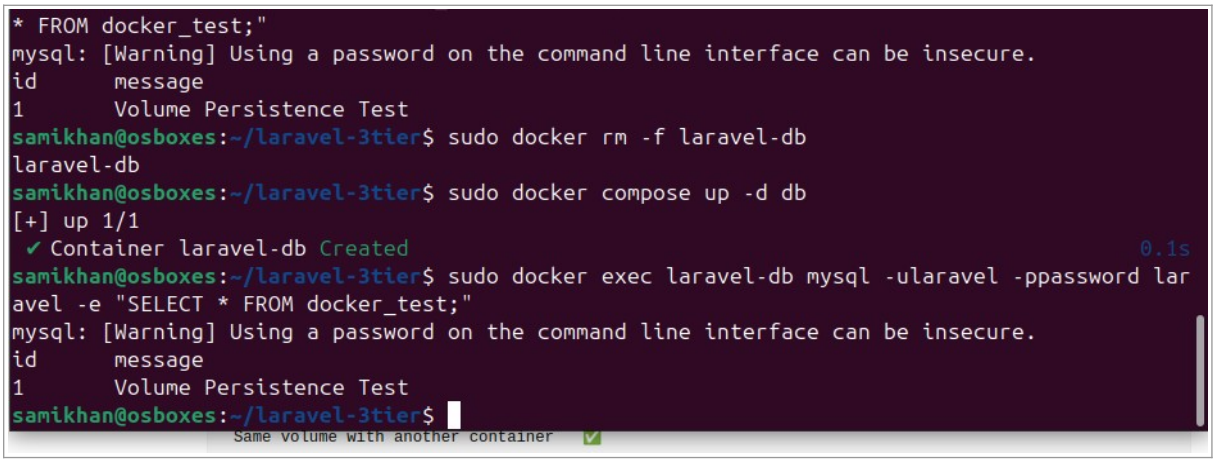


Figure 29: The test data still present after recreating the database container, confirming volume persistence.

Conclusion

This lab demonstrated a complete, practical Docker workflow: installing Docker Engine and Docker Compose from the official repository on Ubuntu, containerizing a three-tier Laravel application consisting of an Nginx frontend, a PHP-FPM backend, and a MySQL database, and verifying the resulting system end-to-end. The application was confirmed to work through the browser, the containers were shown to communicate over Docker's internal bridge network using automatic service-name DNS resolution, and the database container's data was shown to persist independently of the container's own lifecycle through the use of a named Docker volume. Together, these steps cover the core DevOps skills of image building, multi-container orchestration with Compose, inter-container networking, and persistent storage.